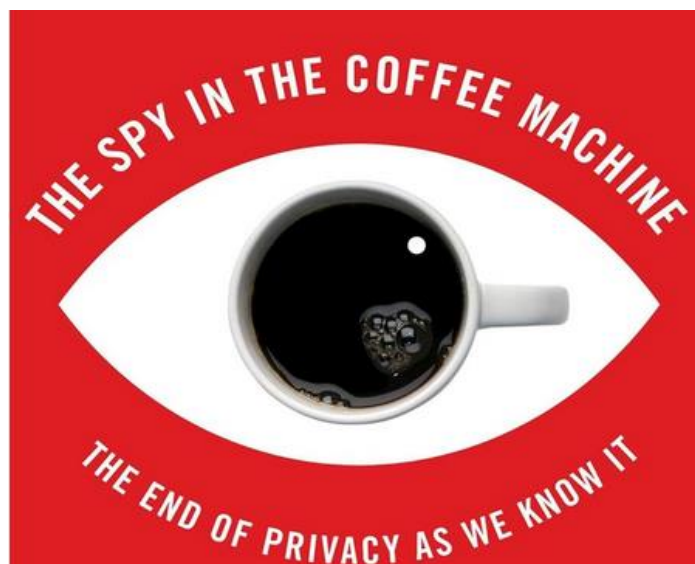


GROUPE N°5

WORKSHOP
Opération **Caft-ière**




myDiL

MULLE Matéo , Mecho David , RASAMIMANANA Jason, SANCHEZ Louane

TABLE DES MATIÈRES

[Intérêt du gadget](#)

[Processus de conception](#)

[Exemple de Code :](#)

[Partie WIFI :](#)

[Dictionnaire MORSE :](#)

[Clignotement de la LED :](#)

[Utilisation du BUZZER :](#)

[Utilisation du détecteur d'aimant :](#)

[Envoi de Code MORSE avec le bouton :](#)

[Schéma Organisationnel](#)

[Les améliorations possibles](#)

Intérêt du gadget

La cafetière remplace la machine de la salle de repos d'une organisation infiltrée. Elle fonctionne normalement, mais contient à l'arrière un module de communication qui envoie et reçoit des messages en morse chiffré. Les fonctions secrètes (envoi, réception, mode furtif) ne s'activent qu'avec un badge autorisé.

Pourquoi elle peut sauver James Bond ?

- Permet d'envoyer un signal d'exfiltration ou des coordonnées sans utiliser les canaux surveillés.
- Reste anodine : Un objet quotidien qui ne suscite pas de suspicion.
- Le contrôle par badge évite les activations accidentelles ou les interceptions par des intrus.

Simple, discrète, sécurisée, la cafetière transforme la pause-café en point de contact sûr pour l'agent en toute discussion.

Processus de conception

Nous avons commencé la première heure à réfléchir à une idée de gadget répondant au thème imposé du workshop.

Nous avons premièrement pensé à une cafetière espionne, un gadget posé dans la salle de repos, là où toutes les papeteries passent notamment des informations intéressantes.

Malheureusement, la configuration de transcription d'un micro étant trop complexe à faire sur un tel projet, nous nous sommes rabattus sur l'idée d'un moyen de communication secret anodin, gardant ainsi l'initiative.

C'est revu de nos idées nous à, au final rendu à notre idée de faire une machine à café, nous permettons d'envoyer et de déchiffrer du code morse.

Exemple de Code :

Partie WIFI :

```
// === WiFI AP ===  
const char* ssid = "CAFTIERE";  
const char* password = "12345678";  
ESP8266WebServer server(80);  
  
// === Handlers Web ===  
void handleRoot() {  
    String page = "<!DOCTYPE html><html><head><meta charset='utf-8'><title>Cafetière</title>"  
        "<style>"  
            "body { background-color:#0b0b0b; color:#e0e0e0; font-family:'Courier New', monospace; text-align:center; }"  
            "h1 { color:#ff0000; text-shadow:0 0 10px #ff0000; font-size:40px; margin-top:20px; }"  
            "h2 { color:#0f0; text-shadow:0 0 5px #0f0; }"  
            "input { padding:10px; width:300px; font-size:16px; border:none; border-radius:5px; margin:10px; background-color:#222; color:#0f0"  
            "button { padding:10px 20px; font-size:16px; border:none; border-radius:5px; cursor:pointer; margin:10px; transition:0.3s; }"  
            "#sendBtn { background-color:#ff0000; color:#fff; box-shadow:0 0 15px #f00; }"  
            "#sendBtn:hover { background-color:#a00; }"  
            "#clearBtn { background-color:#0f0; color:#000; box-shadow:0 0 15px #0f0; }"  
            "#clearBtn:hover { background-color:#090; }"  
            "#melodyBtn { background-color:#00f; color:#fff; box-shadow:0 0 15px #00f; }"  
            "#melodyBtn:hover { background-color:#009; }"  
            "pre { background-color:#000; color:#0f0; padding:15px; border:2px solid #0f0; max-width:600px; margin:20px auto; text-align:left;"  
        "</style></head><body>"  
        "<h1>Cafetière</h1>"  
        "<input type='text' id='msgInput' placeholder='Tapez votre message...'"  
        "<button id='sendBtn' onclick='sendMessage()'>Envoyer</button>"  
        "<button id='melodyBtn' onclick='playMelody()'>Jouer la mélodie</button>"  
        "<label style='display:block; margin:10px; font-size:16px;'>"  
        "<input type='checkbox' id='melodySwitch'> Activer mélodie alternative"  
        "</label>"  
        "<hr>"
```

```

void handleLive() {
    String decoded = decodeMorse(messageEnvoye);
    server.send(200, "text/plain", decoded);
}

void handleClear() {
    messageEnvoye = "";
    server.send(200, "text/plain", "OK");
}

void handleMelody() {
    playMelody();
    server.send(200, "text/plain", "Melody played!");
}

void handleSwitch() {
    if (server.hasArg("state")) {
        String state = server.arg("state");
        useAltMelody = (state == "on");
        Serial.println("Switch web état : " + state);
        server.send(200, "text/plain", "OK");
    } else {
        server.send(400, "text/plain", "Paramètre manquant !");
    }
}

```

```
// === Wifi init ===
WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, password, 1, true);
Serial.print("AP démarré, IP : ");
Serial.println(WiFi.softAPIP());

// === Commande serveur ===
server.on("/", handleRoot);
server.on("/send", handleSend);
server.on("/live", handleLive);
server.on("/clear", handleClear);
server.on("/melody", handleMelody);
server.on("/switch", handleSwitch);

server.begin();
Serial.println("Serveur Web démarré");
```

Dictionnaire MORSE :

```
// === Dictionnaire MORSE ===
std::map<char, String> morseDict = {
    {'A', "-.-"}, {'B', "-..."}, {'C', "-.-."}, {'D', "-.."},
    {'E', "."}, {'F', "..-."}, {'G', "--."}, {'H', "...."},
    {'I', ".."}, {'J', ".---"}, {'K', "-.-"}, {'L', "-..."},
    {'M', "--"}, {'N', "-."}, {'O', "---"}, {'P', "-.-."},
    {'Q', "---."}, {'R', "-.-"}, {'S', "..."}, {'T', "-"},
    {'U', "-.-"}, {'V', "...-"}, {'W', "-.-"}, {'X', "-.-.-"},
    {'Y', "-.-.-"}, {'Z', "---."},
    {'0', "-----"}, {'1', ".-----"}, {'2', "..----"}, {'3', "...---"},
    {'4', "....-"}, {'5', "....."}, {'6', "-...."}, {'7', "--..."},
    {'8', "---.."}, {'9', "----."},
    {' ', " "}};
```

Clignotement de la LED :

```
// Clignotement LED
void blinkMorse(const String &morse) {
    for (unsigned int i = 0; i < morse.length(); i++) {
        char symbol = morse[i];
        if (symbol == '.') {
            digitalWrite(LED1, HIGH);
            delay(COURT);
            digitalWrite(LED1, LOW);
            delay(COURT);
        } else if (symbol == '-') {
            digitalWrite(LED1, HIGH);
            delay(LONG);
            digitalWrite(LED1, LOW);
            delay(COURT);
        } else if (symbol == ' ') {
            delay(GAP);
        }
    }
}
```

Utilisation du BUZZER :

```
// === Mélodies James Bond ===
void playMelody() {
    int melody[] = {1109, 1245, 1245, 1245, 1245, 1109};
    int duration[] = {187, 93, 93, 187, 375, 187, 187};
    for (int i = 0; i < 37; i++) {
        tone(BUZZER, melody[i], duration[i]);
        delay(duration[i] * 1.3);
    }
    noTone(BUZZER);
}

// === Mélodies Totally spies
void playMelody2() {
    int melody2[] = {659, 554, 554, 494, 554, 659};
    int duration[] = {139, 93, 232, 93, 93, 187};
    for (int i = 0; i < 6; i++) {
        tone(BUZZER, melody2[i], duration[i]);
        delay(duration[i] * 1.3);
    }
    noTone(BUZZER);
}
```

Utilisation du détecteur d'aimant :

```
// === Etat MAGNET ===  
int magnetState = digitalRead(MAGNET);  
if (messagePret && magnetState == LOW) {  
    Serial.println("Lecture du message en Morse...");  
    digitalWrite(LEDV, LOW);  
    String morseMessage = trad(message);  
    blinkMorse(morseMessage);  
    messagePret = false;  
}
```

Envoi de Code MORSE avec le bouton :

```
// === BUTTON MORSE ===
static unsigned long pressStart = 0;
static unsigned long lastChange = 0;
static bool isPressed = false;

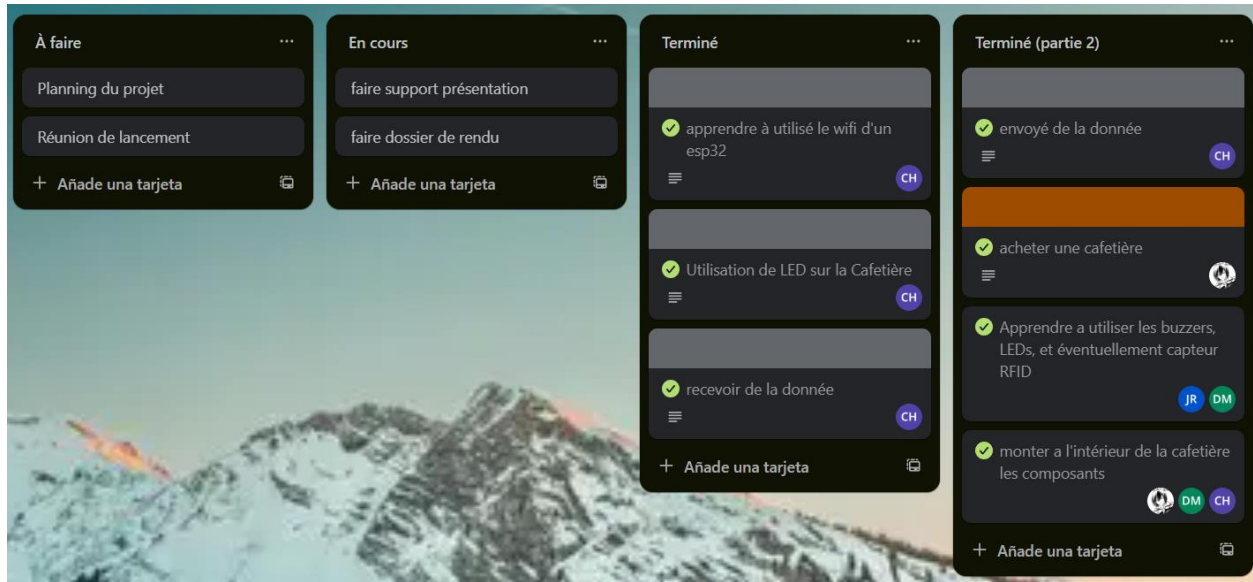
// === Etat BUTTON ===
int buttonState = digitalRead(BUTTON);

if (buttonState == LOW && !isPressed && (millis() - lastChange > DEBOUNCE)) {
    pressStart = millis();
    isPressed = true;
    lastChange = millis();
    digitalWrite(LED_R, HIGH);
}

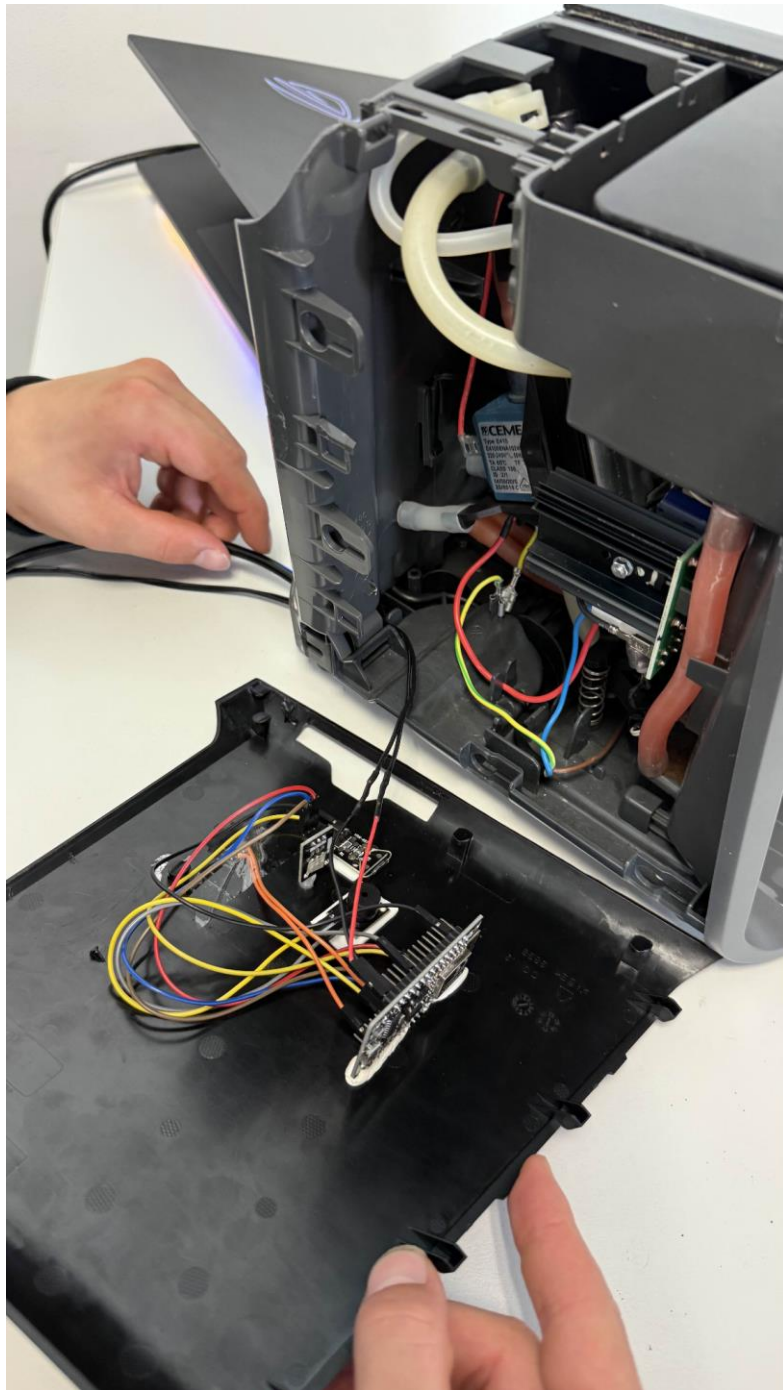
if (buttonState == HIGH && isPressed && (millis() - lastChange > DEBOUNCE)) {
    unsigned long duration = millis() - pressStart;
    isPressed = false;
    lastChange = millis();
    digitalWrite(LED_R, LOW);

    if (duration < LONG) {
        currentLetter += ".";
    } else {
        currentLetter += "-";
    }
    lastRelease = millis();
}
```

Schéma Organisationnel



Au départ, nous nous étions répartis les tâches : Louane prenait en charge le PowerPoint, Matéo développait la partie principale, et Jason et David s'initient aux capteurs RFID. En constatant qu'il était impossible d'intégrer le code RFID à la partie principale, nous nous sommes regroupés pour finir le programme ensemble et avons renoncé à lancer la diffusion du message en morse dès la détection de la bonne carte. Nous avons finalement choisi de déclencher la diffusion du message après la détection d'un aimant.



Les messages morse envoyés entre l'agent et l'espion peuvent être vu en allant sur le réseau secret de la cafétéria.

The image shows a web application interface with a black background. At the top, the word "Cafetière" is written in red. Below it, there is a text input field with the placeholder "Tapez votre message...". To the right of the input field are two buttons: a red "Envoyer" button and a blue "Jouer la mélodie" button. Below these buttons is a checkbox that is checked, followed by the text "Activer mélodie alternative". A horizontal line separates this section from the one below. The section below has the heading "Message en temps réel de l'agent sur place :" in green. Underneath the heading is a large, empty rectangular box with a green border. At the bottom of this section is a green "Effacer" button.

Les améliorations possibles

Pour les améliorations qu'on aurait pu faire serait de faire en sorte que notre idée pour faire en sorte que la RFID puisse se faire d'une manière ou d'une autre. Par exemple, une fois une carte est détectée par le capteur, un compartiment secret dans la cafetière s'ouvre, ou que le message se diffuse grâce à ce capteur.

ANNEXE

1. Photo de la caft-ière fini

